C PROGRAMMING — ASSIGNMENT 1

Task 9: Implement and Test the Billing Program

Marks: 4 | Total assignment: 50 marks

Implementation

This is a complete single-file billing program using integer paise. It validates the two readings, supports one 16-bit rollover, computes the slab charge, and prints the bill. The supplied tariff example assumes a current reading lower than the previous reading is a rollover; in a production system, a separate status/elapsed-time check should distinguish rollover from a mistaken input.

Test table

Case	Previous	Current	Units	Expected bill
Zero use	100	100	0	Rs 50.00
First slab	1000	1050	50	Rs 50.00
Boundary 100	1000	1100	100	Rs 50.00
Boundary 200	1000	1200	200	Rs 275.00
Above 200	1000	1350	350	Rs 950.00
Rollover	65400	120	256	Rs 527.00
Invalid letters	abc	120	—	Reject
Negative reading	-1	120	—	Reject

Five-bill hand calculation

For consumptions 0, 100, 150, 200 and 350 units, the bills are Rs 50.00, Rs 50.00, Rs 162.50, Rs 275.00 and Rs 950.00. Their total is Rs 1,487.50. This is a hand-calculated expected total and can be compared with program output.

Alternative slab loop

A loop over s slab definitions takes O(s) time per bill and requires O(s) tariff storage. Fixed if statements use O(1) time and no slab-array storage, but changing a tariff structure requires editing code. A table-driven loop is easier to extend to more slabs.

C Source Code / Implementation

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdlib.h>
#include <errno.h>

#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE1_PAISE 0LL
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL

static uint32_t consumption(uint16_t previous, uint16_t current) {
    return (uint16_t)(current - previous);
}
static int64_t compute_bill(uint32_t units) {
    int64_t energy = 0;
    if (units > SLAB2_UNITS) energy += (int64_t)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS) {
        uint32_t second = (units < SLAB2_UNITS ? units : SLAB2_UNITS) - SLAB1_UNITS;
        energy += (int64_t)second * RATE2_PAISE;
    }
    if (units <= SLAB1_UNITS) energy += (int64_t)units * RATE1_PAISE;
    return FIXED_CHARGE_PAISE + energy;
}
static int parse_reading(const char *prompt, uint16_t *out) {
    char line[128], *end;
    long v;
    printf("%s", prompt);
    if (!fgets(line, sizeof line, stdin)) return 0;
    errno = 0; v = strtol(line, &end, 10);
    while (*end == ' ' || *end == '\t' || *end == '\n') ++end;
    if (errno || end == line || *end != '\\0' || v < 0 || v > 65535) return 0;
    *out = (uint16_t)v;
    return 1;
}
int main(void) {
    uint16_t previous, current;
    if (!parse_reading("Previous reading (0..65535): ", &previous) ||
        !parse_reading("Current reading (0..65535): ", &current)) {
        fprintf(stderr, "Invalid input: enter integer readings from 0 to 65535.\n");
```

```
        return 1;
    }
    uint32_t units = consumption(previous, current);
    int64_t p = compute_bill(units);
    printf("Units consumed: %" PRIu32 "\n", units);
    printf("Bill: Rs %" PRId64 ".%02" PRId64 "\n", p / 100, p % 100);
    return 0;
}
```

Submission note

Compile and run the code on your own system, save actual terminal output/screenshots, and include them in your report. Review the code and be prepared to explain it during the viva. Any machine-specific measurements must come from your own execution.